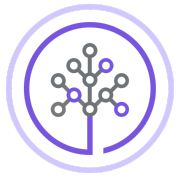


Lab: Symmetric Encryption Using AES



Skills
Network

Estimated Time Needed: 30 minutes

Overview:

In this lab, you will learn how to use OpenSSL to encrypt files using the AES (Advanced Encryption Standard) symmetric encryption algorithm and decrypt them back to their original form. AES provides fast and secure encryption, widely used for protecting sensitive data.

Learning Objectives:

After completing this lab, you will be able to:

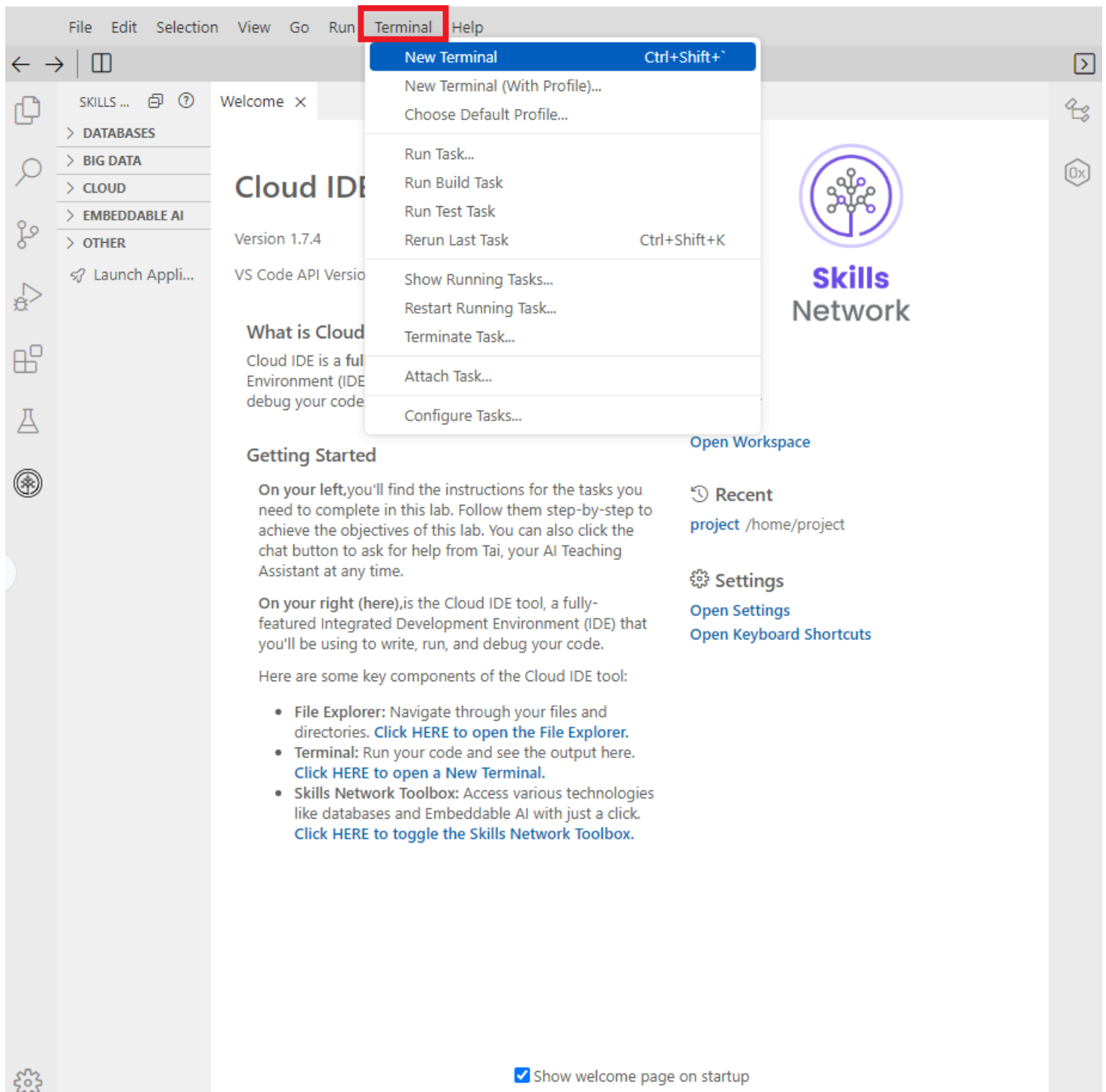
- Generate a secret key for AES encryption
- Encrypt files using AES encryption
- Decrypt files encrypted using AES

Prerequisites (Optional):

Familiarity with using the Linux command prompt.

Initializing the Lab Environment

[Click Terminal to Open New Terminal](#)



Click New Terminal

File Edit Selection View Go Run Terminal Help

SKILLS ... Welcome x

> DATABASES

> BIG DATA

> CLOUD

> EMBEDDABLE AI

> OTHER

Launch Appli...

Cloud IDE

Version 1.7.4

VS Code API Version: 1.88.1

What is Cloud IDE?

Cloud IDE is a **fully-featured** Integrated Development Environment (IDE) that you'll be using to write, run, and debug your code.

Getting Started

On your left, you'll find the instructions for the tasks you need to complete in this lab. Follow them step-by-step to achieve the objectives of this lab. You can also click the chat button to ask for help from Tai, your AI Teaching Assistant at any time.

On your right (here), is the Cloud IDE tool, a fully-featured Integrated Development Environment (IDE) that you'll be using to write, run, and debug your code.

Here are some key components of the Cloud IDE tool:

- File Explorer: Navigate through your files and

Skills Network

Start

New File...

Open

Open Workspace

Recent

project /home/project

Settings

Open Settings

Open Keyboard Shortcuts

```
theia@theia-manishkr: /home/project x
```

```
theia@theia-manishkr: /home/project$
```

Copy and paste the commands into the terminal for the remaining steps.

Step 1: Create a Test File

```
echo "This is a sample file for AES encryption lab." > test_file.txt
```

Command Description

Command/Option	Description
echo	Outputs a string of text.
> test_file.txt	Redirects the text into a file named test_file.txt.

Step 2: Generate a Secret Key for AES

Generate a random 256-bit (32-byte) key for AES encryption:

```
openssl rand -base64 32 > aes_key.bin
```

Command Description

Command/Option	Description
openssl rand	Generates random data.
-base64	Encodes the random data in Base64 format, making it easier to store.
32	Specifies the number of bytes (32 bytes = 256 bits).
> aes_key.bin	Redirects the generated key into a file named aes_key.bin.

Step 3: Encrypt the File Using AES

Encrypt the file using the generated AES key:

```
openssl enc -aes-256-cbc -salt -in test_file.txt -out encrypted_file.bin -pass file:aes_key.bin
```

Command Description

Command/Option	Description
openssl enc	Uses OpenSSL’s symmetric encryption utility.

Command/Option	Description
-aes-256-cbc	Specifies the AES algorithm with 256-bit encryption in CBC mode.
-salt	Adds a random salt to increase encryption strength and prevent attacks.
-in test_file.txt	Specifies the input file to be encrypted.
-out encrypted_file.bin	Specifies the output file for the encrypted data.
-pass file:aes_key.bin	Uses the generated key from aes_key.bin for encryption.

Step 4: Verify the Encrypted File

Check the contents of the encrypted file to confirm that it is no longer in a readable format:

```
cat encrypted_file.bin
```

Command Description

Command/Option	Description
cat	Displays the contents of a file.
encrypted_file.bin	The file with the encrypted binary data.

You should see unreadable binary data, confirming that the file has been encrypted.

Step 5: Decrypt the Encrypted File

Decrypt the encrypted file back to its original content using the same key:

```
openssl enc -d -aes-256-cbc -in encrypted_file.bin -out decrypted_file.txt -pass file:aes_key.bin
```

Command Description

Command/Option	Description
openssl enc -d	Specifies that the operation is decryption.
-aes-256-cbc	Uses the AES algorithm in the same mode as encryption.

Command/Option	Description
-in encrypted_file.bin	Specifies the encrypted file to be decrypted.
-out decrypted_file.txt	Specifies the output file for the decrypted text.
-pass file:aes_key.bin	Uses the same key from aes_key.bin to decrypt the file.

Step 6: Verify the Decrypted File

Finally, check the contents of the decrypted file to confirm that it matches the original:

```
cat decrypted_file.txt
```

Command Description

Command/Option	Description
cat	Displays the contents of a file.
decrypted_file.txt	The file with the decrypted data.

You should see the original text: “This is a sample file for AES encryption lab.”

Exercises

Exercise 1: Encrypt and Decrypt a Small Text File Using AES

Objective: Learn to encrypt and decrypt a small text file using AES symmetric encryption.

Task Details:

Step 1: Create a Short Message:

► Hint!

Step 2: Generate an AES Key:

► Hint!

Step 3: Encrypt the File Using AES:

► Hint!

Step 4: Decrypt the Encrypted File:

► Hint!

Exercise 2: Encrypt and Decrypt a Credentials File Using AES

Objective: Encrypt a file containing sensitive credentials using AES and decrypt it.

Task Details:

Step 1: Create a Credentials File:

► Hint!

Step 2: Generate an AES Key and IV (Initialization Vector):

► Hint!

► Hint!

Step 3: Encrypt the Credentials File Using AES:

► Hint!

Step 4: Decrypt the Encrypted File:

► Hint!

Conclusion

In this lab, you successfully used OpenSSL to encrypt and decrypt files using the AES encryption algorithm. You generated a secure AES key, applied it to encrypt the test file, and successfully decrypted it back to its original form. By following these steps, you learned the fundamentals of symmetric encryption, where the same key is used for both encryption and decryption, allowing for secure protection of sensitive data.

Author(s)

Dr. Manish Kumar

© IBM Corporation. All rights reserved.